



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Developer Guide

C/C++

Version 1.23

March 2026

CONFIDENTIAL

Table of Contents

Revision History	4
Abbreviations	5
1 Introduction	6
1.1 Document Purpose	6
2 Development Environment Setup	7
3 Developing Your First Program	8
3.1 Step 1: Initialize the Toolkit	8
3.2 Step 2: List Connected Readers	8
3.3 Step 3: Connect to a Reader	9
3.4 Step 4: Read Public Data	9
3.5 Step 5: Disconnect and Cleanup	10
4 Common Recipes	11
4.1 Generating a Request ID	11
4.2 PIN Encoding	11
4.3 Checking Card Status	14
4.4 Fingerprint Authentication	15
4.5 PIN Reset	17
4.6 Digital Signing (PAdES)	18
4.7 NFC Card Connection	19
4.8 PKI Authentication	19
5 Toolkit API Reference	21
5.1 Method: Initialize	21
5.2 Method: Cleanup	21
5.3 Method: GetToolkitVersion	21
5.4 Method: GetDeviceID	22
5.5 Method: GetDataProtectionKey	22
5.6 Method: ParseMRZData	23
5.7 Method: RegisterDevice	23
5.8 Method: GetConfigCertificateExpiryDate	23
5.9 Method: GetLicenseExpiryDate	24
5.10 Method: ListReaders	24
5.11 Method: Connect	24
5.12 Method: GetReaderWithEmiratesID	25
5.13 Method: GetConnectedReaderAndSerialNumber	25
5.14 Method: Disconnect	26
5.15 Method: GetCardVersion	26
5.16 Method: SetNfcAuthenticationParams	26
5.17 Method: PrepareRequest	27
5.18 Method: GetCSN	27
5.19 Method: ReadPublicData	27
5.20 Method: ReadPublicDataEF	28
5.21 Method: ParseEFData	29
5.22 Method: CleanupEFDataMemory	29
5.23 Method: GetPKICertificates	29
5.24 Method: CheckCardStatus	30

5.25 Method: GetFingerIndex	30
5.26 Method: AuthenticateBiometricOnServer	31
5.27 Method: AuthenticateCardAndBiometric	31
5.28 Method: ResetPIN	32
5.29 Method: ResetPINWithoutAuthenticateBiometric	32
5.30 Method: UnblockPIN	33
5.31 Method: AuthenticatePKI	33
5.32 Method: SignData	34
5.33 Method: SignChallenge	34
5.34 Method: VerifySignature	35
5.35 Method: ReadFamilyBookData	36
5.36 Method: FreeMemory	36
5.37 Method: GetStatusMessage	37
5.38 Method: ValidateToolkitResponse	37
5.39 Method: PadesSign	38
5.40 Method: PadesVerify	39
5.41 Method: XadesSign	39
5.42 Method: XadesVerify	41
5.43 Method: CadesSign	41
5.44 Method: CadesVerify	42
5.45 Method: CadesSignEx	43
5.46 Method: PadesSignEx	44
5.47 Method: XadesSignEx	44
5.48 Method: ConnectEx	45
5.49 Method: DisconnectEx	45
5.50 Method: ReadPublicDataEx	46
5.51 Method: ResetPINEx	46
5.52 Method: UnblockPINEx	46
5.53 Method: GetInterfaceType	47
5.54 Method: GetSMMMode	47
5.55 Method: GetCPLCData	48
5.56 Method: VerifyCertificateUsingCRL	48
5.57 Method: VerifyCertificateUsingOCSP	48

Revision History

Version	Date	Description of Changes
1.0	22nd Feb, 2017	Beta Release 1
1.1	17th Apr, 2017	Beta Release 2
1.2	9th Aug, 2017	Beta Release 3
1.3	23rd Feb, 2018	Release Version 1.0.0
1.4	30th Mar, 2018	Release Version 1.0.3
1.5	28th May, 2018	Release Version 1.0.5
1.6	12th Jun, 2018	Release Version 1.0.6
1.7	19th Sept, 2018	Release Version 1.0.8
1.8	12th Dec, 2018	Release Version 1.0.9
1.9	16th Feb, 2019	Release Version 1.0.13
1.10	19th Mar, 2019	Release Version 1.1.0
1.11	03rd Apr, 2019	Release Version 1.2.0
1.12	13th May, 2019	Release Version 1.3.0
1.13	07th June, 2019	Release Version 1.4.0
1.14	31st July, 2019	Release Version 1.4.3; Included API's for Card Update functionality.
1.15	04th Mar, 2020	Release Version 1.4.5
1.16	04th Apr, 2020	Release Version 1.4.6
1.17	02nd Nov, 2020	Release Version 1.4.8
1.18	24th May, 2021	Release Version 1.4.9
1.19	04th Oct, 2021	Release Version 2.0.2
1.20	12th Apr, 2022	Release Version 2.0.3
1.21	13th Jun, 2022	Release Version 2.0.5
1.22	20th Dec, 2023	Release version 3.0.0
1.23	Mar 2026	Toolkit v3.0.5. Added 27 missing API methods. Improved code samples.

Abbreviations

Abbreviation	Description
ICA	Identity & Citizenship Authority
SPs	Service Providers
VG	Validation Gateway
MRZ	Machine Readable Zone
NFC	Near Field Communication
PDF	Portable Document Format
XML	Extensible Markup Language
CMS	Cryptographic Message Syntax
PAdES	PDF Advanced Electronic Signatures
CAAdES	CMS Advanced Electronic Signatures
XAdES	XML Advanced Electronic Signatures
EF	Elementary File
CAAdES	CMS Advanced Electronic Signatures
PAdES	PDF Advanced Electronic Signatures
XAdES	XML Advanced Electronic Signatures

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Document Purpose

This document is a developer's guide for building C/C++ applications that integrate the ICP ID Card Toolkit. It covers environment setup, a quick-start tutorial, and the complete API reference for all 64+ Toolkit functions.

2. Development Environment Setup

The following environment setup is based on Visual Studio on Windows. The setup may vary slightly when using different versions or platforms.

1. Add the `EIDAToolkit/include` directory to the include paths in your project settings to access the SDK header files.
2. Add the `EIDAToolkit/lib` directory path to the linker settings and add `EIDAToolkit.lib` to the list of libraries.
3. The Toolkit library and device plugin libraries (for any ID card reader or fingerprint reader devices used) must be in the application executable directory or in the system PATH.

Note: There are different libraries for 32-bit and 64-bit architectures, found in the `x86` and `x64` subdirectories respectively. On Linux, link against `libEIDAToolkit.so`. On macOS, link against `libEIDAToolkit.a` (static library).

3. Developing Your First Program

The following example shows the complete sequence of API calls required to develop an application using the Toolkit SDK for C/C++. This example reads the public data from a connected Emirates ID Card.

Refer to the Toolkit API Reference (Section 4) for details of all available services. A sample C/C++ program is provided with the SDK.

3.1. Step 1: Initialize the Toolkit

The `Initialize` method establishes the Toolkit context. It takes two parameters:

- `in_process_mode`: Set to `TRUE` to load the Toolkit DLL directly (in-process), or `FALSE` to communicate via the Toolkit Agent service.
- `config_params`: JSON string with configuration parameters. Refer to the Programmer's Reference for the full list of configuration parameters.

```
#include "EIDAToolkitApi.h"
#include "EIDAToolkitTypes.h"
#include "EIDAToolkitError.h"

ETSTATUS status;
char *config = "{\"config_directory\":\"C:/EIDAToolkit/config\"}";

// Initialize the Toolkit in in-process mode
status = Initialize(TRUE, config);
if (status != ETSTATUS_SUCCESS) {
    printf("Initialize failed: %s\n", GetStatusMessage(status));
    return 1;
}
```

3.2. Step 2: List Connected Readers

Retrieve a list of connected smartcard readers. The Toolkit allocates memory for the reader list; free it with `FreeMemory` when done.

```
char *readers = NULL;
unsigned int readers_len = 0;
unsigned int reader_count = 0;

status = ListReaders(&readers, &readers_len, &reader_count);
if (status != ETSTATUS_SUCCESS) {
    printf("ListReaders failed\n");
    Cleanup();
    return 1;
}

// readers is a null-separated list of reader names
// reader_count contains the number of readers found
printf("Found %d reader(s)\n", reader_count);
```


3.3. Step 3: Connect to a Reader

Connect to a reader using one of the names returned by `ListReaders`. On success, a connection handle is returned for use in subsequent API calls.

```
unsigned int connection_handle = 0;

// Connect using the first reader name
status = Connect(readers, &connection_handle);
if (status != ETSTATUS_SUCCESS) {
    printf("Connect failed\n");
    FreeMemory(readers);
    Cleanup();
    return 1;
}
```

3.4. Step 4: Read Public Data

Read public data from the connected ID card. The method takes flags to control which data categories are returned. Generate a Request ID (40-byte random, Base64-encoded) before each service call.

```
char *request_id = generate_request_id(); // Your implementation
char *response = NULL;
unsigned int response_len = 0;

status = ReadPublicData(
    connection_handle,
    request_id,
    TRUE, // read_non_modifiable_data
    TRUE, // read_modifiable_data
    TRUE, // read_photo
    FALSE, // read_signature_image
    FALSE, // read_address
    &response,
    &response_len);

if (status == ETSTATUS_SUCCESS) {
    // response contains digitally signed XML with the public data
    printf("Public data read successfully (%u bytes)\n", response_len);

    // Validate the response signature and request ID
    int vg_status = 0;
    char *error_message = NULL;
    unsigned int error_message_len = 0;
    VGRESPOSE_DATA response_data = {0};

    status = ValidateToolkitResponse(
        TRUE, // in_process_mode
        (char *)response, // signed XML response from VG
        NULL, 0, // cert_data, cert_data_len
        // (NULL = use config certificates)
        NULL, 0, // cert_chain, cert_chain_len
        // (NULL = use config certificate chain)
        FALSE, // perform_chain_validation
    );
}
```

```
        &vg_status,           // VG response status code
        &error_message,      // error description (free with FreeMemory)
        &error_message_len,  // error description length
        &response_data       // parsed response data
    );

    if (status != ETSTATUS_SUCCESS) {
        printf("Validation failed: %s\n", GetStatusMessage(status));
        FreeMemory(error_message);
        return;
    }
}
```

3.5. Step 5: Disconnect and Cleanup

Always disconnect the card and clean up resources when done.

```
// Free the response buffer
FreeMemory(response);
FreeMemory(readers);

// Disconnect the card reader
Disconnect(connection_handle);

// Release all Toolkit resources
Cleanup();
```

4. Common Recipes

This section provides complete code samples for common Toolkit operations.

4.1. Generating a Request ID

Every Toolkit service call requires a unique Request ID – a 40-byte cryptographically random value, Base64-encoded. A new Request ID must be generated for each request.

```
#include <openssl/rand.h>
#include <openssl/bio.h>
#include <openssl/evp.h>
#include <openssl/buffer.h>

#define REQUEST_ID_RAW_LEN 40

// Generate a 40-byte random Request ID, Base64-encoded.
// Caller must free the returned string with free().
char *generate_request_id()
{
    unsigned char raw[REQUEST_ID_RAW_LEN];
    BIO *b64, *mem;
    BUF_MEM *buf;

    // Generate 40 cryptographically random bytes
    if (RAND_bytes(raw, REQUEST_ID_RAW_LEN) != 1) {
        return NULL;
    }

    // Base64 encode
    b64 = BIO_new(BIO_f_base64());
    mem = BIO_new(BIO_s_mem());
    b64 = BIO_push(b64, mem);
    BIO_set_flags(b64, BIO_FLAGS_BASE64_NO_NL);
    BIO_write(b64, raw, REQUEST_ID_RAW_LEN);
    BIO_flush(b64);
    BIO_get_mem_ptr(b64, &buf);

    char *request_id = (char *)malloc(buf->length + 1);
    memcpy(request_id, buf->data, buf->length);
    request_id[buf->length] = '\0';

    BIO_free_all(b64);
    return request_id;
}
```

Note: On Windows, you can also use `CryptGenRandom()` or `BCryptGenRandom()` instead of OpenSSL for random number generation.

4.2. PIN Encoding

The PIN must be encrypted before sending to the Toolkit. The complete flow is:

1. Get the VG data protection public key using `GetDataProtectionKey`
2. Get a request handle using `PrepareRequest`
3. Construct plaintext: 8-byte request handle + PIN digits
4. RSA-encrypt with VG public key (PKCS#1 padding)
5. Base64-encode the 256-byte encrypted result

```
#include <openssl/rsa.h>
#include <openssl/bn.h>
#include <openssl/bio.h>
#include <openssl/evp.h>
#include <openssl/buffer.h>
#include <string.h>

#define RSA_KEY_SIZE 256 // 2048-bit RSA = 256 bytes output

// Helper: Base64 decode (returns decoded length)
int base64_decode(const char *input, int input_len,
                 unsigned char *output, int output_max)
{
    BIO *b64 = BIO_new(BIO_f_base64());
    BIO *mem = BIO_new_mem_buf(input, input_len);
    mem = BIO_push(b64, mem);
    BIO_set_flags(mem, BIO_FLAGS_BASE64_NO_NL);
    int decoded_len = BIO_read(mem, output, output_max);
    BIO_free_all(mem);
    return decoded_len;
}

// Helper: Base64 encode (caller must free result)
char *base64_encode(const unsigned char *input, int input_len)
{
    BIO *b64 = BIO_new(BIO_f_base64());
    BIO *mem = BIO_new(BIO_s_mem());
    b64 = BIO_push(b64, mem);
    BIO_set_flags(b64, BIO_FLAGS_BASE64_NO_NL);
    BIO_write(b64, input, input_len);
    BIO_flush(b64);
    BUF_MEM *buf;
    BIO_get_mem_ptr(b64, &buf);
    char *result = (char *)malloc(buf->length + 1);
    memcpy(result, buf->data, buf->length);
    result[buf->length] = '\0';
    BIO_free_all(b64);
    return result;
}

// Encrypt a PIN for Toolkit API calls.
// Call GetDataProtectionKey() and PrepareRequest() first.
// Returns Base64-encoded encrypted PIN. Caller must free().
char *encrypt_pin(
    const char *pin_digits,
    const char *request_handle_b64,
    const char *modulus_hex,
    const char *exponent_hex)
{

```

```

unsigned char plaintext[24]; // 8 (handle) + up to 16 (PIN)
unsigned char encrypted[RSA_KEY_SIZE];
int pin_len = (int)strlen(pin_digits);

// Step 1: Base64-decode the Request Handle (8 bytes)
unsigned char handle_raw[8];
int handle_len = base64_decode(
    request_handle_b64,
    (int)strlen(request_handle_b64),
    handle_raw, 8);

if (handle_len != 8) {
    return NULL; // Invalid request handle
}

// Step 2: Build plaintext = Request Handle + PIN digits
memcpy(plaintext, handle_raw, 8);
memcpy(plaintext + 8, pin_digits, pin_len);
int total_len = 8 + pin_len;

// Step 3: Build RSA public key from modulus and exponent
RSA *rsa = RSA_new();
BIGNUM *n = NULL, *e = NULL;
BN_hex2bn(&n, modulus_hex);
BN_hex2bn(&e, exponent_hex);
RSA_set0_key(rsa, n, e, NULL);

// Step 4: RSA encrypt with PKCS#1 v1.5 padding
int enc_len = RSA_public_encrypt(
    total_len, plaintext, encrypted,
    rsa, RSA_PKCS1_PADDING);

RSA_free(rsa);

if (enc_len != RSA_KEY_SIZE) {
    return NULL; // Encryption failed
}

// Step 5: Base64-encode the 256-byte encrypted result
return base64_encode(encrypted, RSA_KEY_SIZE);
}

```

Usage: Call GetDataProtectionKey and PrepareRequest before encrypting:

```

// Get VG data protection public key
unsigned char *dpk = NULL;
unsigned int dpk_len = 0;
char *modulus = NULL, *exponent = NULL;
unsigned int mod_len = 0, exp_len = 0;

status = GetDataProtectionKey(
    &dpk, &dpk_len,
    &modulus, &mod_len,
    &exponent, &exp_len);

```

```
if (status != ETSTATUS_SUCCESS) {
    printf("Failed to get data protection key\n");
    return;
}

// Prepare the service request
char *request_id = generate_request_id();
char *request_handle = NULL;
unsigned int handle_len = 0;

status = PrepareRequest(
    connection_handle,
    request_id,
    &request_handle,
    &handle_len);

if (status != ETSTATUS_SUCCESS) {
    printf("PrepareRequest failed\n");
    goto cleanup;
}

// Encrypt the PIN using VG public key and request handle
char *encrypted_pin = encrypt_pin(
    "1234",           // PIN digits (4-16 digits)
    request_handle,   // Base64 request handle from PrepareRequest
    modulus,          // RSA modulus from GetDataProtectionKey
    exponent);        // RSA exponent from GetDataProtectionKey

if (encrypted_pin == NULL) {
    printf("PIN encryption failed\n");
    goto cleanup;
}

// encrypted_pin is now ready for use with
// ResetPIN, UnblockPIN, AuthenticatePKI, SignData, etc.

cleanup:
free(encrypted_pin);
free(request_id);
FreeMemory(request_handle);
FreeMemory(dpk);
FreeMemory(modulus);
FreeMemory(exponent);
```

4.3. Checking Card Status

Validate whether a connected card is active, expired, or revoked.

```
char *request_id = generate_request_id();
char *response = NULL;
unsigned int response_len = 0;

status = CheckCardStatus(
```

```

    connection_handle,
    request_id,
    &response,
    &response_len);

if (status == ETSTATUS_SUCCESS) {
    // Response XML contains <CardStatus> element:
    // "CardValid", "CardExpired", "CardRevoked", "CardNotRegistered"
    printf("Card status response: %.*s\n", response_len, response);
}
else if (status == ETSTATUS_CARD_EXPIRED) {
    printf("Card has expired\n");
}
else if (status == ETSTATUS_CARD_REVOKED) {
    printf("Card has been revoked\n");
}

FreeMemory(response);
free(request_id);

```

4.4. Fingerprint Authentication

Authenticate a cardholder using their fingerprint. The complete flow is:

1. Call `GetFingerIndex` to retrieve the stored finger IDs and indexes from the card
2. Call `PrepareRequest` to establish a secure context
3. Call `AuthenticateBiometricOnServer` with the finger index from step 1

```

// Step 1: Get the finger indexes stored on the card
unsigned char first_finger_id = 0;
unsigned char first_finger_index = 0;
unsigned char second_finger_id = 0;
unsigned char second_finger_index = 0;

status = GetFingerIndex(
    connection_handle,
    &first_finger_id,
    &first_finger_index,
    &second_finger_id,
    &second_finger_index);

if (status != ETSTATUS_SUCCESS) {
    printf("GetFingerIndex failed\n");
    return;
}

printf("First finger: id=%d, index=%d\n",
    first_finger_id, first_finger_index);
printf("Second finger: id=%d, index=%d\n",
    second_finger_id, second_finger_index);

// Use the first finger for authentication
unsigned int finger_index = first_finger_index;

// Step 2: Prepare the request

```

```
char *request_id = generate_request_id();
char *request_handle = NULL;
unsigned int handle_len = 0;

status = PrepareRequest(
    connection_handle,
    request_id,
    &request_handle,
    &handle_len);

if (status != ETSTATUS_SUCCESS) {
    printf("PrepareRequest failed\n");
    goto cleanup;
}

// Step 3: Authenticate using the finger index from the card
char *response = NULL;
unsigned int response_len = 0;
int sensor_timeout = 30; // seconds

status = AuthenticateBiometricOnServer(
    connection_handle,
    request_id,
    finger_index, // from GetFingerIndex
    sensor_timeout,
    &response,
    &response_len);

if (status == ETSTATUS_SUCCESS) {
    printf("Biometric authentication successful\n");

    // Validate the response signature
    int vg_status = 0;
    char *error_message = NULL;
    unsigned int error_message_len = 0;
    VGRESPONSE_DATA response_data = {0};
    status = ValidateToolkitResponse(
        TRUE, (char *)response,
        NULL, 0, NULL, 0, FALSE,
        &vg_status, &error_message, &error_message_len,
        &response_data);
    FreeMemory(error_message);
}
else if (status == ETSTATUS_BIO_AUTH_FAILED) {
    printf("Fingerprint did not match\n");
}

cleanup:
FreeMemory(response);
FreeMemory(request_handle);
free(request_id);
```


Note: The finger index values returned by `GetFingerIndex` correspond to the fingerprints stored on the card during enrollment. Always use these values rather than hardcoding a finger index.

4.5. PIN Reset

Reset a cardholder's PIN. This requires biometric authentication first (performed internally by the Toolkit). The new PIN must be encrypted using the VG data protection key.

```
// Step 1: Get the data protection key
unsigned char *dpk = NULL;
unsigned int dpk_len = 0;
char *modulus = NULL, *exponent = NULL;
unsigned int mod_len = 0, exp_len = 0;

status = GetDataProtectionKey(
    &dpk, &dpk_len,
    &modulus, &mod_len,
    &exponent, &exp_len);

// Step 2: Prepare the request
char *request_id = generate_request_id();
char *request_handle = NULL;
unsigned int handle_len = 0;

PrepareRequest(
    connection_handle,
    request_id,
    &request_handle,
    &handle_len);

// Step 3: Encrypt the new PIN
char *encrypted_pin = encrypt_pin(
    "1234",           // new PIN digits
    request_handle,   // Base64 request handle
    modulus, exponent); // VG public key

// Step 4: Reset the PIN (includes biometric auth)
int attempts_left = 0;

status = ResetPIN(
    encrypted_pin,
    strlen(encrypted_pin),
    RIGHT_INDEX,    // finger index for biometric auth
    0,              // finger_id (0 = auto)
    30,            // sensor timeout in seconds
    &attempts_left);

if (status == ETSTATUS_SUCCESS) {
    printf("PIN reset successful\n");
}
else {
    printf("PIN reset failed, attempts left: %d\n", attempts_left);
}
```

```
}

// Cleanup
free(encrypted_pin);
free(request_id);
FreeMemory(request_handle);
FreeMemory(dpk);
FreeMemory(modulus);
FreeMemory(exponent);
```

4.6. Digital Signing (PAdES)

Digitally sign a PDF document using the cardholder's signing certificate on the ID card.

```
// Step 1: Prepare the signing context
SIGNING_CONTEXT ctx;
memset(&ctx, 0, sizeof(SIGNING_CONTEXT));

ctx.signature_level = BASELINE_B;      // Basic signature
ctx.packaging_mode = ENVELOPED;       // Embedded in PDF

// Encrypt the PIN and copy to context
char *enc_pin = encrypt_pin("1234", request_handle,
                             modulus, exponent);
strcpy(ctx.pin, enc_pin);
free(enc_pin);

// Step 2: Configure visible signature (optional)
PADES_SIGN_PARAMS params;
memset(&params, 0, sizeof(PADES_SIGN_PARAMS));

params.sign_visible = TRUE;
strcpy(params.signer_name, "John Doe");
params.page_number = 1;
params.x = 50;
params.y = 50;
params.w = 200;
params.h = 80;
params.name_position = BOTTOM;

// Step 3: Sign the PDF
int attempts_left = 0;

status = PadesSign(
    &ctx,
    "C:/documents/contract.pdf",
    "C:/documents/contract_signed.pdf",
    &params,
    &attempts_left);

if (status == ETSTATUS_SUCCESS) {
    printf("PDF signed successfully\n");
}
else {
```

```
    printf("Signing failed, attempts left: %d\n", attempts_left);  
}
```

4.7. NFC Card Connection

Connect to an Emirates ID Card via NFC. For V2 cards this additionally requires supplying MRZ data or card number + date of birth + expiry date so the Toolkit can perform NFC authentication.

```
// Step 1: Connect to the NFC reader  
unsigned int connection_handle = 0;  
status = Connect("NFC Reader", &connection_handle);  
  
if (status != ETSTATUS_SUCCESS) {  
    printf("NFC connect failed\n");  
    return;  
}
```

Note: V2 cards only: SetNfcAuthenticationParams is only required for V2 cards. V3, V4, and V5 cards handle NFC authentication automatically — this step can be skipped for those card versions.

```
// Step 2: Set NFC authentication parameters  
// Option A: Using card details  
status = SetNfcAuthenticationParams(  
    connection_handle,  
    "784199012345678", // card_number  
    "560412",          // dob (YYMMDD)  
    "291116",          // expiry_date (YYMMDD)  
    NULL);             // mrz_data (NULL when using card details)  
  
// Option B: Using MRZ string (alternative)  
// status = SetNfcAuthenticationParams(  
//     connection_handle,  
//     NULL, NULL, NULL,  
//     "I<AZE...full MRZ string...");  
  
if (status == ETSTATUS_SUCCESS) {  
    // Now proceed with ReadPublicData, CheckCardStatus, etc.  
    printf("NFC authentication successful\n");  
}
```

4.8. PKI Authentication

Authenticate the cardholder using PIN with PKI challenge-response. This also verifies the revocation status of the card's authentication certificate.

```
// Encrypt the PIN  
char *encrypted_pin = encrypt_pin(  
    "1234", request_handle,  
    modulus, exponent);
```

```
int attempts_left = 0;
char *response = NULL;
unsigned int response_len = 0;

status = AuthenticatePKI(
    encrypted_pin,
    strlen(encrypted_pin),
    &attempts_left,
    &response,
    &response_len);

if (status == ETSTATUS_SUCCESS) {
    printf("PKI authentication successful\n");
    // response contains signed authentication result
}
else if (status == ETSTATUS_PINVERIFICATION_FAILED) {
    printf("Wrong PIN, %d attempts left\n", attempts_left);
}

free(encrypted_pin);
FreeMemory(response);
```

5. Toolkit API Reference

This section documents all Toolkit API functions. All functions return ETSTATUS. Use ETSTATUS_CODE(status) to extract the error code. Memory allocated by the Toolkit (returned via char** or unsigned char** parameters) must be freed using FreeMemory().

5.1. Method: Initialize

Initialize the Emirates ID Card Toolkit

```
ETSTATUS Initialize(  
    BOOL in_process_mode,  
    char *config_params);
```

Parameters:

Parameter	Description
in_process_mode [in]	TRUE for in-process mode (DLL loaded directly), FALSE for agent mode (communicates via Agent service)
config_params [in]	JSON string with configuration, or NULL for defaults. In agent mode, pass the agent URL. In in-process mode, pass a JSON string containing at minimum the config directory path:

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.2. Method: Cleanup

Cleanup the Emirates ID Card Toolkit

```
ETSTATUS Cleanup();
```

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.3. Method: GetToolkitVersion

Get the Toolkit Version

```
ETSTATUS GetToolkitVersion(  
    int *major_version,  
    int *minor_version,  
    int *patch_version);
```

Parameters:

Parameter	Description
major_version [out]	EIDAToolkit major version number
minor_version [out]	EIDAToolkit minor version number

Parameter	Description
patch_version [out]	EIDAToolkit patch version number

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.4. Method: GetDeviceID

Toolkit API to Get Device ID

```
ETSTATUS GetDeviceID(  
    char **device_id,  
    unsigned int *device_id_len);
```

Parameters:

Parameter	Description
device_id [out]	Unique Device ID string
device_id_len [out]	Device ID string length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.5. Method: GetDataProtectionKey

Get data protection key

```
ETSTATUS GetDataProtectionKey(  
    unsigned char **data_protection_key,  
    unsigned int *key_len,  
    char **modulus,  
    unsigned int *modulus_len,  
    char **exponent,  
    unsigned int *exponent_len);
```

Parameters:

Parameter	Description
data_protection_key [out]	Data protection public key
key_len [out]	Data protection public key length
modulus [out]	Data protection public key modulus
modulus_len [out]	Data protection public key modulus length
exponent [out]	Data protection public key public exponent
exponent_len [out]	Data protection public key public exponent length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.6. Method: ParseMRZData

Parse MRZ data

```
ETSTATUS ParseMRZData(  
    char *mrz,  
    MRZ_DATA *mrz_data);
```

Parameters:

Parameter	Description
mrz [in]	Data from the Machine Readable Zone (MRZ) of the card
mrz_data [out]	Parsed MRZ string attributes

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.7. Method: RegisterDevice

Registers a device with Validation Gateway (VG) against the Service Provider (SP) license

```
ETSTATUS RegisterDevice(  
    char *user_id,  
    char *password,  
    char *device_reference_id,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
user_id [in]	Authorised Service Provider (SP) user id (encrypted)
password [in]	Encrypted password of the user
device_reference_id [in]	Device registration reference number
response [out]	XML response from VG. Caller must free with FreeMemory().
response_len [out]	Response length in bytes

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.8. Method: GetConfigCertificateExpiryDate

Get config certificate expiry date

```
ETSTATUS GetConfigCertificateExpiryDate(CONFIG_CERT_EXPIRY_DATE  
    *config_cert_expiry_date);
```

Parameters:

Parameter	Description
config_cert_expiry_date [out]	Structure to receive config certificate expiry date

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.9. Method: GetLicenseExpiryDate

Get license expiry date

```
ETSTATUS GetLicenseExpiryDate(  
    char **expiry_date,  
    unsigned int *expiry_date_len);
```

Parameters:

Parameter	Description
expiry_date [out]	Toolkit license expiry date
expiry_date_len [out]	Toolkit license expiry date length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.10. Method: ListReaders

Gets the List of readers connected.

```
ETSTATUS ListReaders(  
    char **readers_list,  
    unsigned int *list_length,  
    unsigned int *readers_count);
```

Parameters:

Parameter	Description
readers_list [out]	Return buffer of readers list.
list_length [out]	Readers name list length in bytes.
readers_count [out]	Number of reader names in the list.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.11. Method: Connect

Establish connection with the smartcard in the reader

```
ETSTATUS Connect(  
    char *reader_name,  
    unsigned int *connection_handle);
```


Parameters:

Parameter	Description
reader_name [in]	Smartcard reader name (from ListReaders())
connection_handle [out]	Handle to smartcard connection. Pass this handle to subsequent card operations.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.12. Method: GetReaderWithEmiratesID

Get first reader with Emirates ID Card inserted

```
ETSTATUS GetReaderWithEmiratesID(  
    char *reader_name,  
    int *reader_name_max);
```

Parameters:

Parameter	Description
reader_name [in]	Connected reader name
reader_name_max [inout]	Reader name length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.13. Method: GetConnectedReaderAndSerialNumber

Get first reader with Emirates ID Card inserted and its serial number

```
ETSTATUS GetConnectedReaderAndSerialNumber(  
    char **reader_name,  
    unsigned int *reader_name_len,  
    char **reader_serial_number,  
    unsigned int *reader_serial_len);
```

Parameters:

Parameter	Description
reader_name [out]	Connected reader name
reader_name_len [out]	Reader name length
reader_serial_number [out]	Connected reader serial number
reader_serial_len [out]	Reader serial number length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.14. Method: Disconnect

Disconnect smartcard session

```
ETSTATUS Disconnect(unsigned int connection_handle);
```

Parameters:

Parameter	Description
connection_handle [in]	Handle to smartcard connection

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.15. Method: GetCardVersion

Toolkit API to Get card version

```
ETSTATUS GetCardVersion(  
    int connection_handle,  
    unsigned char *card_version);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
card_version [out]	Emirates ID Card version

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.16. Method: SetNfcAuthenticationParams

Set NFC Authentication params for the Emirates ID Card

```
ETSTATUS SetNfcAuthenticationParams(  
    int connection_handle,  
    char *card_number,  
    char *date_of_birth,  
    char *expiry_date,  
    char *mrz_data);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID card connection handle
card_number [in]	Card Number (CN)
date_of_birth [in]	Card holder Date of Birth (DoB)
expiry_date [in]	Emirates ID Card expiry date

Parameter	Description
mrz_data [in]	Data from the Machine Readable Zone (MRZ) of the card

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.17. Method: PrepareRequest

Prepare service request

```
ETSTATUS PrepareRequest(  
    unsigned int connection_handle,  
    char *request_id,  
    char **request_handle,  
    unsigned int *request_handle_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Handle to an Emirates ID Card connection
request_id [in]	Application-generated random string for request tracking
request_handle [out]	8-byte request handle encoded in base64 format. Caller must free with FreeMemory().
request_handle_len [out]	Length of the request handle

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.18. Method: GetCSN

Gets the Card Serial Number (CSN)

```
ETSTATUS GetCSN(  
    int connection_handle,  
    char **csn,  
    unsigned int *csn_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
csn [out]	Card serial number (CSN)
csn_len [out]	Card Serial Number (CSN) length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.19. Method: ReadPublicData

Read public data from the Emirates ID Card

```
ETSTATUS ReadPublicData(  
    unsigned int connection_handle,  
    char *request_id,  
    int non_modifiable_data,  
    int modifiable_data,  
    int photography,  
    int signature_image,  
    int address,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
request_id [in]	Application generated random request ID
non_modifiable_data [in]	TRUE to read name, DOB, nationality, etc.
modifiable_data [in]	TRUE to read phone, email, etc.
photography [in]	TRUE to read cardholder photo (JPEG, base64 in response)
signature_image [in]	TRUE to read handwritten signature image
address [in]	TRUE to read home and work address
response [out]	XML response string from VG. Caller must free with FreeMemory().
response_len [out]	Response length in bytes

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.20. Method: ReadPublicDataEF

Toolkit API to read public data EF from the Emirates ID Card

```
ETSTATUS ReadPublicDataEF(  
    unsigned int connection_handle,  
    PUBLIC_DATA_EF_TYPE public_data_ef_type,  
    BOOL validate_signature,  
    unsigned char **ef_data,  
    unsigned int *ef_data_length);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
public_data_ef_type [in]	Enumeration of public data EF type to read corresponding data from Emirates ID Card

Parameter	Description
validate_signature [in]	Flag to perform EF signature validation.
ef_data [out]	Pointer to receive EF data
ef_data_length [out]	EF data length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.21. Method: ParseEFData

EIDA Toolkit API to parse EF data

```
ETSTATUS ParseEFData(  
    unsigned char *ef_data,  
    unsigned int ef_data_len,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
ef_data [in]	Buffer containing the EF data
ef_data_len [in]	EF data length
response [out]	Parsed EF data
response_len [out]	Parsed EF data length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.22. Method: CleanupEFDataMemory

Cleanup EF data memory

```
ETSTATUS CleanupEFDataMemory();
```

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.23. Method: GetPKICertificates

Toolkit API to export certificates

```
ETSTATUS GetPKICertificates(  
    char *pin,  
    unsigned int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.24. Method: CheckCardStatus

This function checks the card status with the help of Validation Gateway (VG)

```
ETSTATUS CheckCardStatus(  
    unsigned int connection_handle,  
    char *request_id,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
request_id [in]	Application random number
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.25. Method: GetFingerIndex

Get stored finger indexes from the Emirates ID Card

```
ETSTATUS GetFingerIndex(  
    int connection_handle,  
    unsigned char *first_finger_id,  
    unsigned char *first_finger_index,  
    unsigned char *second_finger_id,  
    unsigned char *second_finger_index);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle

Parameter	Description
first_finger_id [out]	First finger id
first_finger_index [out]	First finger index
second_finger_id [out]	Second finger id
second_finger_index [out]	Second finger index

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.26. Method: AuthenticateBiometricOnServer

Perform biometric authentication on server (Validation Gateway - VG)

```
ETSTATUS AuthenticateBiometricOnServer(  
    unsigned int connection_handle,  
    char *request_id,  
    unsigned int finger_index,  
    int sensor_timeout,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
request_id [in]	Application random number
finger_index [in]	Index of finger to be used for bio. authentication
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.27. Method: AuthenticateCardAndBiometric

Perform check card status and biometric authentication on server (Validation Gateway - VG)

```
ETSTATUS AuthenticateCardAndBiometric(  
    unsigned int connection_handle,  
    char *request_id,  
    unsigned int finger_index,  
    int sensor_timeout,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
request_id [in]	Application random number
finger_index [in]	Index of finger to be used for bio. authentication
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.28. Method: ResetPIN

Reset PIN on Emirates ID Card

```
ETSTATUS ResetPIN(  
    char *pin,  
    unsigned int pin_len,  
    unsigned int finger_index,  
    unsigned int finger_id,  
    int sensor_timeout,  
    int *attempts_left);
```

Parameters:

Parameter	Description
pin [in]	New PIN to set on the Emirates ID Card in encrypted format
pin_len [in]	Length of the encrypted PIN
finger_index [in]	Index of finger to be used for bio. authentication
finger_id [in]	Finger ID to be used for bio. authentication
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
attempts_left [out]	Number of attempts left before the card is blocked.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.29. Method: ResetPINWithoutAuthenticateBiometric

Reset PIN on Emirates ID Card without biometric authentication

```
ETSTATUS ResetPINWithoutAuthenticateBiometric(  
    char *pin,  
    unsigned int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```


Parameters:

Parameter	Description
pin [in]	New PIN to set on the Emirates ID Card in encrypted format
pin_len [in]	Length of the encrypted PIN
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.30. Method: UnblockPIN

Toolkit API to unblock Emirates ID card PKI PIN

```
ETSTATUS UnblockPIN(  
    char *pin,  
    unsigned int pin_len,  
    unsigned int finger_index,  
    unsigned int finger_id,  
    int sensor_timeout,  
    int *attempts_left);
```

Parameters:

Parameter	Description
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
finger_index [in]	Index of finger to be used for bio. authentication
finger_id [in]	Finger identifier
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
attempts_left [out]	Number of attempts left before the card is blocked.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.31. Method: AuthenticatePKI

Perform Emirates ID Card PKI authentication

```
ETSTATUS AuthenticatePKI(  
    char *pin,  
    unsigned int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.32. Method: SignData

Digitally sign data or hash provided

```
ETSTATUS SignData(  
    unsigned char *data,  
    unsigned int data_len,  
    BOOL data_hash,  
    char* pin,  
    unsigned int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
data [in]	Plain data or hash to sign
data_length [in]	Length of plain data or hash provided
data_hash [in]	TRUE if hash/FALSE if plain data
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.33. Method: SignChallenge

Digitally sign challenge or hash provided

```
ETSTATUS SignChallenge(  
    unsigned char *challenge,  
    unsigned int challenge_len,  
    BOOL data_hash,  
    char* pin,  
    unsigned int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
challenge [in]	Plain data or hash to sign
challenge_len [in]	Length of the challenge
data_hash [in]	TRUE if hash/FALSE if plain data
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.34. Method: VerifySignature

Verify digital signature

```
ETSTATUS VerifySignature(  
    unsigned char *data,  
    unsigned int data_len,  
    unsigned char *signature,  
    unsigned int signature_len,  
    unsigned char *certificate,  
    unsigned int certificate_len,  
    BOOL data_hash);
```

Parameters:

Parameter	Description
data [in]	Plain data or hash for which signature is verified
data_len [in]	Plain data or hash length
signature [in]	Digital signature of the data

Parameter	Description
signature_len [in]	Digital signature length
certificate [in]	Certificate data by which data is signed.
certificate_len [in]	Certificate data length.
data_hash [in]	TRUE if hash/FALSE if plain data.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.35. Method: ReadFamilyBookData

Toolkit API to read family book data from the card

```
ETSTATUS ReadFamilyBookData(  
    char *pin,  
    int pin_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
pin [out]	Emirates ID Card PIN encrypted
pin_len [out]	The encrypted PIN length
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.36. Method: FreeMemory

Free a memory buffer allocated by this module

```
ETSTATUS FreeMemory(void *buffer);
```

Parameters:

Parameter	Description
buffer [in]	Memory buffer to free (returned by a Toolkit API)

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.37. Method: GetStatusMessage

```
const char* GetStatusMessage(  
ETSTATUS status);
```

Parameters:

Parameter	Description
status [in]	Toolkit status value.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.38. Method: ValidateToolkitResponse

EIDA Toolkit API to verify Toolkit response

```
ETSTATUS ValidateToolkitResponse(  
    BOOL in_process_mode,  
    char *vg_response,  
    unsigned char *cert_data,  
    unsigned int cert_data_len,  
    unsigned char *cert_chain,  
    unsigned int cert_chain_len,  
    BOOL chain_validation,  
    int *status,  
    char **error_message,  
    unsigned int *error_message_len,  
    VGRESPONSE_DATA *response_data);
```

Parameters:

Parameter	Description
in_process_mode [in]	TRUE if toolkit is used as in-process module
vg_response [in]	Validation Gateway (VG) response
cert_data [in]	Validation Gateway (VG) response signing certificate
cert_data_len [in]	Certificate data length
cert_chain [in]	Validation Gateway (VG) response signing certificate chain
cert_chain_len [in]	Certificate chain length
chain_validation [in]	Flag (TRUE/FALSE) indicating chain validation is required or not
status [out]	Signature validation status(-1 not set / 1 set)
error_message [out]	Error message
error_message_len [out]	Length of error message
response_data [out]	Parsed VG response attributes

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.39. Method: PadesSign

Perform digital signing of PDF document following PDF Advanced Electronic Signature (PAdES) standard

```
ETSTATUS PadesSign(  
    SIGNING_CONTEXT *signing_context,  
    char *pdf_file_path,  
    char *signed_pdf_file_path,  
    PADES_SIGN_PARAMS *pades_params,  
    int *attempts_left);
```

Parameters:

Parameter	Description
signing_context [in]	Toolkit digital signature context
pdf_file_path [in]	Path of the input pdf document
signed_pdf_file_path [in]	Path of the signed pdf document
pades_params [in]	Additional PAdES specific parameters
attempts_left [out]	Number of attempts left before the card is blocked.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

Example:

```
SIGNING_CONTEXT ctx;  
memset(&ctx, 0, sizeof(SIGNING_CONTEXT));  
  
// Encrypt PIN and set in context  
char *enc_pin = encrypt_pin("1234", request_handle,  
                             modulus, exponent);  
strcpy(ctx.pin, enc_pin);  
free(enc_pin);  
  
ctx.signature_level = BASELINE_B;  
ctx.packaging_mode = ENVELOPED;  
  
// Configure visible signature  
PADES_SIGN_PARAMS params;  
memset(&params, 0, sizeof(PADES_SIGN_PARAMS));  
params.sign_visible = TRUE;  
strcpy(params.signer_name, "John Doe");  
params.page_number = 1;  
params.x = 50; params.y = 50;  
params.w = 200; params.h = 80;  
params.name_position = BOTTOM;  
  
// Sign the PDF
```

```
int attempts_left = 0;
status = PadesSign(
    &ctx,
    "C:/docs/contract.pdf",
    "C:/docs/contract_signed.pdf",
    &params,
    &attempts_left);

if (status == ETSTATUS_SUCCESS) {
    printf("PDF signed successfully\n");
}
else {
    printf("Signing failed, attempts left: %d\n",
        attempts_left);
}
```

5.40. Method: PadesVerify

Perform digital signature verification of PDF document following PDF Advanced Electronic Signature (PAdES) standard

```
ETSTATUS PadesVerify(
    VERIFICATION_CONTEXT *verification_context,
    char *pdf_file_path,
    char **verification_report,
    unsigned int *report_len);
```

Parameters:

Parameter	Description
verification_context [in]	Digital signature verification context
pdf_file_path [in]	Path of the input pdf document to verify
verification_report [out]	Verification report
report_len [out]	Verification report length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.41. Method: XadesSign

Perform digital signing of XML document following XML Advanced Electronic Signature (XAdES) standard

```
ETSTATUS XadesSign(
    SIGNING_CONTEXT *signing_context,
    char *xml_file_path,
    char *signed_xml_file_path,
    unsigned char **signature,
    unsigned int *signature_len,
    int *attempts_left);
```

Parameters:

Parameter	Description
signing_context [in]	Toolkit digital signature context
xml_file_path [in]	Path of the input xml document
signed_xml_file_path [in]	Path of the signed xml document
signature [out]	signature data if packaging mode is Detached
signature_len [out]	signature data length
attempts_left [out]	Number of attempts left before the card is blocked.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

Example — enveloped mode:

```
SIGNING_CONTEXT ctx;
memset(&ctx, 0, sizeof(SIGNING_CONTEXT));

char *enc_pin = encrypt_pin("1234", request_handle,
                           modulus, exponent);
strcpy(ctx.pin, enc_pin);
free(enc_pin);

ctx.signature_level = BASELINE_B;
ctx.packaging_mode = ENVELOPED;

int attempts_left = 0;

status = XadesSign(
    &ctx,
    "C:/docs/invoice.xml",
    "C:/docs/invoice_signed.xml",
    NULL, // signature (not used for enveloped)
    NULL, // signature_len
    &attempts_left);
```

Example — detached mode:

```
ctx.packaging_mode = DETACHED;
unsigned char *signature = NULL;
unsigned int signature_len = 0;

status = XadesSign(
    &ctx,
    "C:/docs/invoice.xml",
    NULL, // no output file for detached
    &signature,
    &signature_len,
    &attempts_left);
```



```
if (status == ETSTATUS_SUCCESS) {  
    // Save detached signature to file  
    FILE *f = fopen("C:/docs/invoice.xml.sig", "wb");  
    fwrite(signature, 1, signature_len, f);  
    fclose(f);  
    FreeMemory(signature);  
}
```

5.42. Method: XadesVerify

Perform digital signature verification of XML document following XML Advanced Electronic Signature (XAdES) standard

```
ETSTATUS XadesVerify(  
    VERIFICATION_CONTEXT *verification_context,  
    char *xml_file_path,  
    unsigned char *signature,  
    unsigned int signature_len,  
    char **verification_report,  
    unsigned int *report_len);
```

Parameters:

Parameter	Description
verification_context [in]	Digital signature verification context
xml_file_path [in]	Path of the input xml document to verify
signature [in]	signature data if packaging mode is Detached
signature_len [in]	signature data length
verification_report [out]	Verification report
report_len [out]	Verification report length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.43. Method: CadesSign

Perform digital signing of document following CMS Advanced Electronic Signature (CAAdES) standard

```
ETSTATUS CadesSign(  
    SIGNING_CONTEXT *signing_context,  
    char *input_document_path,  
    unsigned char **signature,  
    unsigned int *signature_len,  
    int *attempts_left);
```

Parameters:

Parameter	Description
signature_context [in]	Toolkit digital signature context
input_document_path [in]	Path of the input document
signature [out]	signature data
signature_len [out]	signature data length
attempts_left [out]	Number of attempts left before the card is blocked.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

Example:

```
SIGNING_CONTEXT ctx;
memset(&ctx, 0, sizeof(SIGNING_CONTEXT));

char *enc_pin = encrypt_pin("1234", request_handle,
                             modulus, exponent);
strcpy(ctx.pin, enc_pin);
free(enc_pin);

ctx.signature_level = BASELINE_B;
ctx.packaging_mode = DETACHED;

unsigned char *signature = NULL;
unsigned int signature_len = 0;
int attempts_left = 0;

status = CadesSign(
    &ctx,
    "C:/docs/contract.pdf",
    &signature,
    &signature_len,
    &attempts_left);

if (status == ETSTATUS_SUCCESS) {
    // Save the CAdES signature (PKCS#7/CMS format)
    FILE *f = fopen("C:/docs/contract.pdf.p7s", "wb");
    fwrite(signature, 1, signature_len, f);
    fclose(f);
    FreeMemory(signature);
}
```

Note: CAdES operates in detached mode only. The signature is returned as a separate PKCS#7/CMS blob that must be stored alongside the original document.

5.44. Method: CadesVerify

Perform digital signature verification of document following CMS Advanced Electronic Signature (CAdES) standard

```
ETSTATUS CadesVerify(  
    VERIFICATION_CONTEXT *verification_context,  
    char *input_document_path,  
    unsigned char *signature,  
    unsigned int signature_len,  
    char **verification_report,  
    unsigned int *report_len);
```

Parameters:

Parameter	Description
verification_context [in]	Digital signature verification context
input_document_path [in]	Path of the input document to verify
signature [in]	CAdES signature of the input document
signature_len [in]	Length of the signature bytes
verification_report [out]	Verification report
report_len [out]	Verification report length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.45. Method: CadesSignEx

Perform digital signing of document following CMS Advanced Electronic Signature (CAdES) standard

```
ETSTATUS CadesSignEx(  
    SIGNING_CONTEXT *signing_context,  
    char *input_document_path,  
    unsigned char **signature,  
    unsigned int *signature_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
signature_context [in]	Toolkit digital signature context
input_document_path [in]	Path of the input document
signature [out]	signature data
signature_len [out]	signature data length
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response

Parameter	Description
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.46. Method: PadesSignEx

Perform digital signing of PDF document following PDF Advanced Electronic Signature (PAdES) standard

```
ETSTATUS PadesSignEx(  
    SIGNING_CONTEXT *signing_context,  
    char *pdf_file_path,  
    char *signed_pdf_file_path,  
    PADES_SIGN_PARAMS *pades_params,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
signing_context [in]	Toolkit digital signature context
pdf_file_path [in]	Path of the input pdf document
signed_pdf_file_path [in]	Path of the signed pdf document
pades_params [in]	Additional PAdES specific parameters
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.47. Method: XadesSignEx

Perform digital signing of XML document following XML Advanced Electronic Signature (XAdES) standard

```
ETSTATUS XadesSignEx(  
    SIGNING_CONTEXT *signing_context,  
    char *xml_file_path,  
    char *signed_xml_file_path,  
    unsigned char **signature,  
    unsigned int *signature_len,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
signing_context [in]	Toolkit digital signature context
xml_file_path [in]	Path of the input xml document
signed_xml_file_path [in]	Path of the signed xml document
signature [out]	signature data if packaging mode is Detached
signature_len [out]	signature data length
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.48. Method: ConnectEx

Establish connection with the smartcard (extended)

```
ETSTATUS ConnectEx(  
    char *reader_name,  
    unsigned int *connection_handle);
```

Parameters:

Parameter	Description
reader_name [in]	Smartcard reader name
connection_handle [out]	Handle to smartcard connection

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.49. Method: DisconnectEx

Disconnect smartcard session (extended)

```
ETSTATUS DisconnectEx(unsigned int connection_handle);
```

Parameters:

Parameter	Description
connection_handle [in]	Handle to smartcard connection

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.50. Method: ReadPublicDataEx

Read public data from the card into a flat structure

```
ETSTATUS ReadPublicDataEx(  
    unsigned int connection_handle,  
    PUBLIC_DATA_OBJ *public_data_obj);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
PUBLIC_DATA_OBJ [out]	Structure to receive public data attributes

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.51. Method: ResetPINEx

Reset PIN Ex on Emirates ID Card

```
ETSTATUS ResetPINEx(  
    char *pin,  
    unsigned int pin_len,  
    unsigned int finger_index,  
    unsigned int finger_id,  
    int sensor_timeout,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
pin [in]	New PIN to set on the Emirates ID Card in encrypted format
pin_len [in]	Length of the encrypted PIN
finger_index [in]	Index of finger to be used for bio. authentication
finger_id [in]	Finger ID to be used for bio. authentication
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.52. Method: UnblockPINEx

Toolkit API to unblock Emirates ID card PKI PIN

```
ETSTATUS UnblockPINEx(  
    char *pin,  
    unsigned int pin_len,  
    unsigned int finger_index,  
    unsigned int finger_id,  
    int sensor_timeout,  
    int *attempts_left,  
    char **response,  
    unsigned int *response_len);
```

Parameters:

Parameter	Description
pin [in]	Emirates ID Card PIN encrypted
pin_len [in]	Length of the encrypted PIN
finger_index [in]	Index of finger to be used for bio. authentication
finger_id [in]	Finger identifier
sensor_timeout [in]	Fingerprint sensor capture timeout in seconds
attempts_left [out]	Number of attempts left before the card is blocked.
response [out]	Pointer to receive the response
response_len [out]	Response length

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.53. Method: GetInterfaceType

Retrieves the interface type from plugin context

```
ETSTATUS GetInterfaceType(  
    int connection_handle,  
    int *interface_type);
```

Parameters:

Parameter	Description
connection_handle [in]	Emirates ID Card connection handle
interface_type [out]	Connection interface type.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.54. Method: GetSMMode

Retrieves the SecureMessaging mode

```
ETSTATUS GetSMMode(int *sm_mode);
```

Parameters:

Parameter	Description
sm_mode [out]	Type of Secure Messaging mode

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.55. Method: GetCPLCData

Toolkit API to read Card Production Life Cycle (CPLC) data from card

```
ETSTATUS GetCPLCData(  
    int connection_handle,  
    CPLC_INFO *cplc_info);
```

Parameters:

Parameter	Description
connection_handle [in]	Handle to an Emirates ID Card connection handle
cplc_info [out]	Card Production Life Cycle (CPLC) data

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.56. Method: VerifyCertificateUsingCRL

Verify certificate using crl

```
ETSTATUS VerifyCertificateUsingCRL(  
    unsigned char *crl_path,  
    unsigned int crl_filetype,  
    unsigned char *cert_data,  
    unsigned int cert_data_len);
```

Parameters:

Parameter	Description
crl_path [in]	crl path
crl_filetype [in]	CRL file type (X509_FILETYPE_PEM = 1/ X509_FILETYPE_ASN1 = 2)
cert_data [in]	Certificate data by which data is signed.
cert_data_len [in]	Certificate data length.

Returns: ETSTATUS_SUCCESS on success, error code on failure.

5.57. Method: VerifyCertificateUsingOCSP

Verify certificate using OCSP


```
ETSTATUS VerifyCertificateUsingOCSP(  
    char *ocsp_url,  
    unsigned char *cert_data,  
    unsigned int cert_data_len,  
    unsigned char *issuer_cert_data,  
    unsigned int issuer_cert_len);
```

Parameters:

Parameter	Description
ocsp_url [in]	OCSP url
cert_data [in]	Certificate data for which status is checked.
cert_data_len [in]	Certificate data length.
issuer_cert_data [in]	Issuer Certificate data.
issuer_cert_len [in]	Issuer Certificate data length.

Returns: ETSTATUS_SUCCESS on success, error code on failure.